

9 febrero 2026

Búsqueda o planeación (planning)

Reflex States Variables Logic

Machine learning

Ya no son sistemas reflejos. Ahora son problemas de búsqueda o planeación.

> Modelos dinámicos

- espacio de estado
- modelo de transición
- estado inicial, test de meta, función de costo de camino (path cost)

Una solución es una secuencia de acciones que transforman el estado inicial al estado meta.

> Goal test es el conjunto de est. finales

$S = \{s_1, \dots, s_n\}$ estado $s \in S$ espacio de estado

$|S|$ finito

Modelo transición: $A = \{a_1, \dots, a_n\}$ acciones

$A: S \rightarrow P(A)$ (conjunto potencia de A)
donde $A(s) \subseteq A$ (acciones legales en el estado s)

Costo total: $S \times A \rightarrow \mathbb{R}^+$ (conjunto de todos conjuntos)
costo (s, a) es el costo de aplicar la acción a en el estado s y un estado inicial $s_0 \in S$ y un conjunto de estados finales

> $\text{succ}: S \times A \rightarrow S$

¿Cuál es el problema?

Encontrar un plan: aplicar acciones con costo

$[(s_0, a_0, c_0), (s_1, a_1, c_1), \dots, (s_T, a_T, c_T)]$

> Func. su que me diga si $s \in S_f$ (estado final)
donde:

$s_{i+1} = \text{succ}(s_i, a_i)$

$\text{succ}(s_T, a_T) \in S_f$ {mi último estado

$c_i = c_{i+1} + \text{costo_total}(s_i, a_i)$ {es un estado terminal}

tal que:

CT sea mínimo (minimizar el costo)

> En $T+1$ ya no hay costo, ya es el estado terminal.

Farmmer, Cabbage, Goat, Wolf

$S = \{FCGN \mid F, C, G, N \in \{0, 1\}\}$

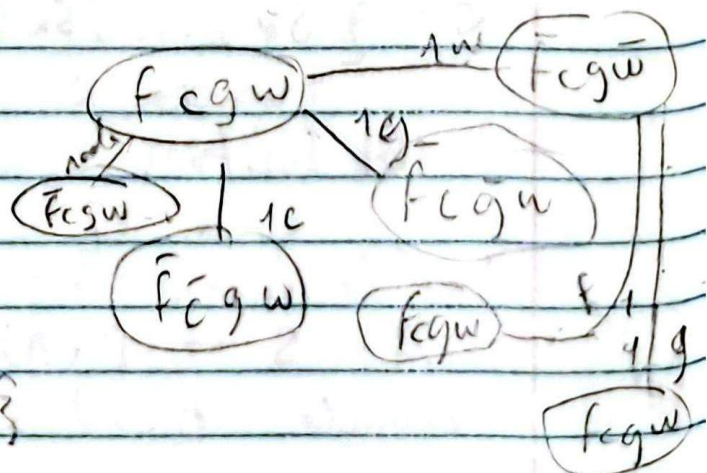
$F = f \quad s_i \leftarrow \bar{f} \rightarrow$

$C = c \quad s_i \leftarrow \bar{c} \rightarrow$

$G = g \quad s_i \leftarrow \bar{g} \rightarrow$

$W = w \quad s_i \leftarrow \bar{w} \rightarrow$

$A = \{c, g, w, \text{move}\}$



10 febrero 2026

```
class SearchPb (Object):
    def __init__(self, S_ini):
        self.s = S_ini
    def acciones(self, s):
        raise NotImplementedError('A desarrollar')
    def successor(self, s, a):
        raise NotImplementedError('A desarrollar')
    def terminal(self, s):
        raise NotImplementedError('A desarrollar')
```

Torres de Hanoi

 $s \in (S_1, \dots, S_n)$
 $S_i \in \{ 'A', 'B', 'C' \}$

```
class TorresHanoi (SearchPb):
    def __init__(self, S0 = ['A'] * 5, Sf = S0[-1] * ['C']):
        self.S0 = S0
        self.Sf = Sf
    def acciones(self, s):
        if any([Si not in ['A', 'B', 'C'] for Si in s]):
            raise ValueError('Estado imposible')
        acciones = { 'AB', 'AC',
                     'BA', 'BC',
                     'CA', 'CB' }
        acciones_locales = [ ]
        for a in acciones:
            if a[0] in s:
```


10 febrero 2026

if $a[0]$ in s :

if $a[1]$ not in s or $s.index[a[0]] < s.index[a[1]]$:

$acciones_legales.append(a)$

> Bonito.

$acciones_legales = [a \text{ in self.acciones if } a[0] \text{ in } s \text{ and } a[1] \text{ not in } s \text{ or } s.index[a[0]] < s.index[a[1]]]$
return

def successor(self, s, a):

$costo_local = 1$

if a not in self.acciones(s):

raise ValueError('—')

$s.next = s.copy()$ o $s[:]$

o $s.find$ $s.next[s.index(a[0]):] = a[1]$

return $s.next, costo_local$

def terminal(self, s):

return $s == self.sf$

> $s0 = s \times [A], sf = s \times [C]$

$torreS = TorresHampi(s0, sf)$

for $_$ in range(5000): $s = s0[:]$ ^{historial} $costo = 0$

$acciones = TorreS.acciones(s)$

$a = random.choice(acciones)$

$s.n, c.f = torreS.sucessor(s, a)$

$costo += c.f$

$s = s.n[:]$ $\rightarrow$ historial.append(a)

if $torreS.terminal(s)$: break

else:

print('Se resolvió a lo menos en {costo} pasos')

print('Las acciones realizadas fueron: {s}')
print('')

print('No se pudo resolver en 5000 pasos')

11 febrero 2026

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

$S = (S_0, \dots, S_{15})$

$S_i \in \{0, \dots, 15\}$

$\text{Card}(S) = \frac{16!}{2}$

15 puzzle "Movermos" el espacio vacío

$A = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$

> Costo-local = 1

class NPuzzle(SearchPb):
 def init(self, n, sf=None):
 self.n = n
 if sf != None and len(sf) != n**2:
 raise ValueError('sf y n no coinciden')
 if sf == None:
 self.s_f = [range(n**2)]
 elif sorted(sf) != [range(n**2)]:
 raise ValueError('sf no es una permutación')
 else:
 self.s_f * sf[:]

def acciones(self, s):
 if not exist(self.a_legales):
 self.a_legales = {}
 for i in range(self.n**2):
 temp = []
 if i // self.n > 0:
 temp.append('↑')

11 febrero 2026

```
if ↑ // self.n < self.n-1:  
    temp.append('↓')  
if i := self.n > 0:  
    temp.append('→')  
if i % self.n < self.n-1  
    temp.append('←')  
self.a-legales[i] = temp[i] @ memoize  
return self.a-legales[s.index(0)] (memoize)
```

def sucesor (self, s, a):

costo-local = 1

if a not in self.acciones(s):

raise ValueError('—')

s_n = s[:] (copio s)

i_vacio = s_n.index(0)

i_otro = (i_vacio + 1 if a == '→' else

i_vacio - 1 if a == '←' else

i_vacio + self.n if a == '↓' else

i_vacio - self.n)

s_n[i_vacio], s_n[i_otro] = s_n[i_otro],
s_n[i_vacio] > swap

return s_n, costo-local

11 febrero 2026

```
def terminal (self, s):  
    return s == self.s_f
```

$$\text{card}(S) = \frac{(n^2)!}{2}$$

busqueda (search Pb, s_0):

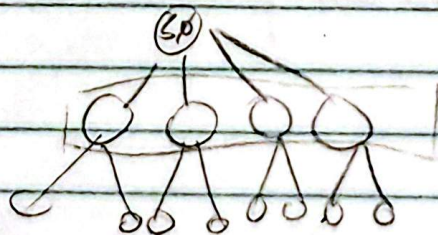
Lo que queremos es un plan.

plan = $[(s_0, a_0, c_0), (s_1, a_1, c_1), (s_2, a_2, c_2), \dots, (s_{T-1}, a_{T-1}, c_{T-1}), (s_T, \text{None}, \text{None})]$

donde s_T = Sucesor (s_{T-1}, a_{T-1}) es un estado terminal.

- La primera vez va a hacer el plan con el agente ~~entorno~~ ~~interno~~ modelo interno.
- Implementa el plan y va verificando que se está siguiendo (s, a, c iguales).
- Si no, se agarrará s como el nuevo s_0 y se genera otro plan.

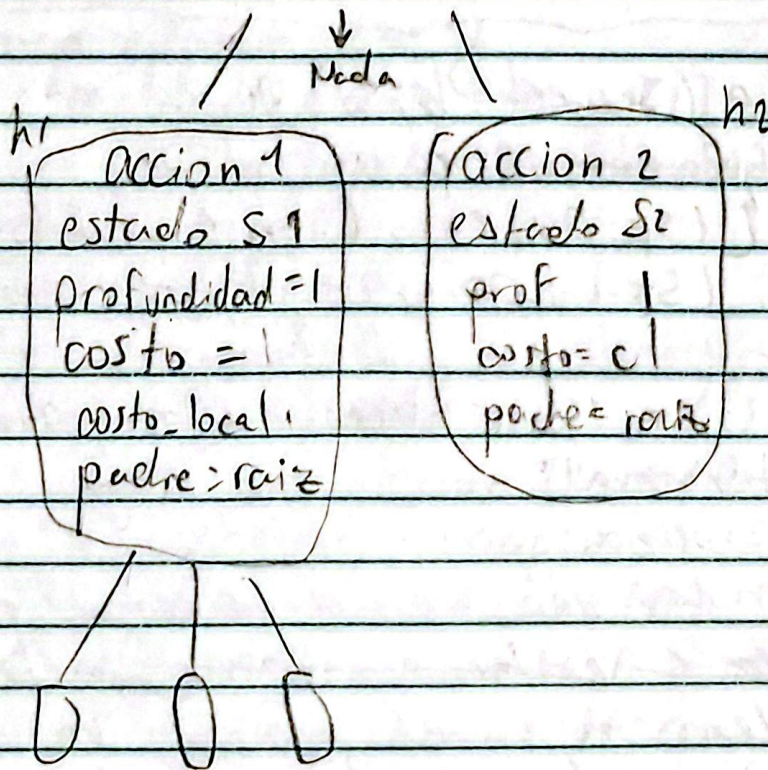
>> Arbol de planes.



¿Cuál es el costo hasta
aca? Me voy por el
menor

12 febrero 2026

raíz { estado = S_0
accion = None
profundidad = 0
padre = None
costo = 0



- ≥ Si mi plan me lleva al terminal de menor costo es el plan óptimo.
- > Si me lleva al terminal no con el menor costo, es un plan admisible.

12 febrero 2026

```
class NodoSearch (Object):  
    def __init__ (self, s, a=None, padre=None,  
                  costo_l=None):
```

```
        self.s = s
```

```
        self.a = a      (accion que llevo al nodo)
```

```
        self.padre = padre
```

```
        self.d = 0 if padre==None else
```

```
                    self.padre.d + 1      (depth pa-  
                                           dre + 1)
```

```
        self.costo = 0 if padre==None else
```

```
                    costo_l + self.padre.costo
```

```
    def expande (self, search_pb):
```

```
        for a in search_pb.acciones(self.s):
```

```
            s_n, costo_l = search_pb.successor
```

```
                (s, a)
```

> Por cada accion que tenga genera
el siguiente y su costo

```
            yield NodoSearch (s_n, a, self,  
                               costo_l)
```

```
Plan = [(S0, a0, c0), (S1, a1, c1) ... (Sn, None, None)]
```


12 febrero 2026

def plan(self):

return [(self.s, None, None)] if
self.padre == None else

self.padre.plan()[i-1]
+ [(self.padre.s, self.a, self.costo),
(self.s, None, None)]

Forma generica

So objeto
problema

def busqueda_generica(s, pb):

frontera > Todos los posibles planes revisa-
bles no revisados

frontera = [NodoSearch(s)]

True while frontera: True if not vacia
[plan = saca_nodo(frontera)]
if pb.terminal(plan.s):
return plan

Agrega
a frontera

for plan_hijo in plan.expande(pb):
[agrega_a_frontera(frontera, plan_hijo)]

return None ← No existe plan